计算机组成原理 Project1 实验报告（Week 2）
8 位定点卷积–激活–池化链运算器设计

鲁昕宁 2414015

2026 年 4 月 26 日

https://github.com/SidneyLu/Computer-Organization-and-Design-NKU-CS/tree/master/project1

1 里程碑 2：链式运算器集成与系统仿真

1.1 串行链式运算器实现

本实验第一组实现了基于串行状态机的链式运算器，完整支持“卷积 → ReLU → 池化”三段链路。

顶层 cnn_compare_base_top 的 Verilog 源码

对应文件：../src/baseline/cnn_compare_base_top.v

```
module cnn_compare_base_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_base u_base (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .use_booth(1'b0),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

模块 cnn_chain_core_base 的 Verilog 源码

对应文件：../src/baseline/cnn_chain_core_base.v

```
module cnn_chain_core_base (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire use_booth,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output reg  done,
    output reg  [31:0] out_flat_u8x4
);
    localparam S_IDLE = 3'd0;
```

```
localparam S_CONV = 3'd1;
localparam S_RELU = 3'd2;
localparam S_POOL = 3'd3;
localparam S_DONE = 3'd4;

reg [2:0] state;
reg [1:0] ox;
reg [1:0] oy;
reg [3:0] k;
reg [3:0] relu_idx;
reg [1:0] pool_idx;

reg signed [19:0] conv_map [0:8];
reg [7:0] relu_map [0:8];

wire [7:0] pix_cur;
wire signed [7:0] ker_cur;
wire signed [15:0] prod_base;
wire signed [15:0] prod_booth;
wire signed [15:0] prod_cur;
wire [7:0] relu_cur;

wire [7:0] p0;
wire [7:0] p1;
wire [7:0] p2;
wire [7:0] p3;
wire [9:0] pool_sum;
wire [7:0] pool_avg;

integer loop_i;
wire [3:0] conv_idx;
wire [5:0] pix_idx;
wire [3:0] base_idx;
wire [1:0] pool_r;
wire [1:0] pool_c;

wire [1:0] ix;
wire [1:0] iy;
assign ix = k % 3;
assign iy = k / 3;
assign conv_idx = oy * 3 + ox;
assign pix_idx = (oy + iy) * 5 + (ox + ix);
assign p0 = relu_map[base_idx];
assign p1 = relu_map[base_idx + 1];
assign p2 = relu_map[base_idx + 3];
assign p3 = relu_map[base_idx + 4];
```

```
assign pool_sum = {2'b00, p0} + {2'b00, p1} + {2'b00, p2} + {2'b00, p3};

assign pix_cur = img_flat_u8x25[pix_idx*8 +: 8];
assign ker_cur = ker_flat_s8x9[k*8 +: 8];

mul8 u_mul8 (
    .a_u8(pix_cur),
    .b_s8(ker_cur),
    .p_s16(prod_base)
);

mul8_booth_wallace u_mul8_booth (
    .a_u8(pix_cur),
    .b_s8(ker_cur),
    .p_s16(prod_booth)
);

assign prod_cur = use_booth ? prod_booth : prod_base;

relu_s20 u_relu (
    .x_s20(conv_map[relu_idx]),
    .y_u8(relu_cur)
);

div4 u_div4 (
    .x_u10(pool_sum),
    .y_u8(pool_avg)
);

assign pool_r = pool_idx[1];
assign pool_c = pool_idx[0];
assign base_idx = pool_r * 3 + pool_c;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= S_IDLE;
        ox <= 2'd0;
        oy <= 2'd0;
        k <= 4'd0;
        relu_idx <= 4'd0;
        pool_idx <= 2'd0;
        done <= 1'b0;
        out_flat_u8x4 <= 32'd0;
        for (loop_i = 0; loop_i < 9; loop_i = loop_i + 1) begin
            conv_map[loop_i] <= 20'sd0;
            relu_map[loop_i] <= 8'd0;
```

```
    end
end else begin
    done <= 1'b0;
    case (state)
        S_IDLE: begin
            if (start) begin
                ox <= 2'd0;
                oy <= 2'd0;
                k <= 4'd0;
                relu_idx <= 4'd0;
                pool_idx <= 2'd0;
                out_flat_u8x4 <= 32'd0;
                for (loop_i = 0; loop_i < 9; loop_i = loop_i + 1)
                ↪  begin
                    conv_map[loop_i] <= 20'sd0;
                    relu_map[loop_i] <= 8'd0;
                end
                state <= S_CONV;
            end
        end
        S_CONV: begin
            conv_map[conv_idx] <= conv_map[conv_idx] +
            ↪  $signed(prod_cur);
            if (k == 4'd8) begin
                k <= 4'd0;
                if (ox == 2'd2) begin
                    ox <= 2'd0;
                    if (oy == 2'd2) begin
                        oy <= 2'd0;
                        state <= S_RELU;
                    end else begin
                        oy <= oy + 2'd1;
                    end
                end else begin
                    ox <= ox + 2'd1;
                end
            end else begin
                k <= k + 4'd1;
            end
        end
        S_RELU: begin
            relu_map[relu_idx] <= relu_cur;
            if (relu_idx == 4'd8) begin
                relu_idx <= 4'd0;
                state <= S_POOL;
            end else begin
```

```
                    relu_idx <= relu_idx + 4'd1;
                end
            end
            S_POOL: begin
                out_flat_u8x4[pool_idx*8 +: 8] <= pool_avg;
                if (pool_idx == 2'd3) begin
                    pool_idx <= 2'd0;
                    state <= S_DONE;
                end else begin
                    pool_idx <= pool_idx + 2'd1;
                end
            end
            S_DONE: begin
                done <= 1'b1;
                state <= S_IDLE;
            end
            default: begin
                state <= S_IDLE;
            end
        endcase
    end
  end
endmodule
```

优化版本的链式运算器（BoothWallace、SIMD、流水线、DSP 版本）源码与仿真结果详见第三部分。

1.2 标准样例与手算结果

系统 testbench `tb_cnn_core` 采用如下标准输入：

- 图像矩阵为从 1 到 25 的顺序数列，按行优先排布为 5×5；
- 卷积核 9 个元素均为 1。

对应的卷积结果为：

$$\begin{bmatrix} 63 & 72 & 81 \\ 108 & 117 & 126 \\ 153 & 162 & 171 \end{bmatrix}$$

ReLU 后保持不变。继续执行 2×2 平均池化，可得到：

$$\begin{bmatrix} 90 & 99 \\ 135 & 144 \end{bmatrix}$$

在 `out_flat_u8x4` 中，该结果被打包为 `32'h9087635A`。

1.3 基础模块系统仿真结果

系统级仿真命令为：

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_vivado_sim.ps1 -Mode
↪ system
```

`tb_cnn_core` 对串行顶层拉起 `start`，并显式检查输出与延迟行为。结果如表 1 所示。

表 1: 基础模块系统仿真结果

实验组	输出结果	延迟周期
串行	`32'h9087635A`	96

XSIM 日志中可以直接看到如下 PASS 信息：

```
CASE=1 PASS cycles=96 out=9087635a
ALL TESTS PASSED.
```

下面给出系统级 testbench 的完整 Verilog 源码。

系统级 testbench tb_cnn_core 的 Verilog 源码

对应文件：../sim/tb_cnn_core.v

```
`timescale 1ns/1ps

module tb_cnn_core;
    reg clk;
    reg rst_n;
    reg start_base;
    reg start_booth;
    reg start_simd;
    reg start_pipe;
    reg start_dsp;
    reg [199:0] img_flat_u8x25;
    reg signed [71:0] ker_flat_s8x9;
    wire done_base;
    wire done_booth;
    wire done_simd;
    wire done_pipe;
    wire done_dsp;
    wire [31:0] out_base_u8x4;
    wire [31:0] out_booth_u8x4;
    wire [31:0] out_simd_u8x4;
    wire [31:0] out_pipe_u8x4;
    wire [31:0] out_dsp_u8x4;

    localparam [31:0] EXPECT = {8'd144, 8'd135, 8'd99, 8'd90};

    cnn_compare_base_top u_base (
        .clk(clk),
        .rst_n(rst_n),
        .start(start_base),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done_base),
        .out_flat_u8x4(out_base_u8x4)
    );

    cnn_compare_booth_top u_booth (
        .clk(clk),
        .rst_n(rst_n),
        .start(start_booth),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done_booth),
        .out_flat_u8x4(out_booth_u8x4)
```

```
);

cnn_compare_booth_simd_top u_simd (
    .clk(clk),
    .rst_n(rst_n),
    .start(start_simd),
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .done(done_simd),
    .out_flat_u8x4(out_simd_u8x4)
);

cnn_compare_booth_simd_pipe_top u_pipe (
    .clk(clk),
    .rst_n(rst_n),
    .start(start_pipe),
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .done(done_pipe),
    .out_flat_u8x4(out_pipe_u8x4)
);

cnn_compare_booth_simd_pipe_dsp_top u_dsp (
    .clk(clk),
    .rst_n(rst_n),
    .start(start_dsp),
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .done(done_dsp),
    .out_flat_u8x4(out_dsp_u8x4)
);

always #5 clk = ~clk;

task load_img_1_to_25;
    integer i;
    begin
        img_flat_u8x25 = 200'd0;
        for (i = 0; i < 25; i = i + 1) begin
            img_flat_u8x25[i*8 +: 8] = i + 1;
        end
    end
endtask

task load_kernel_all_ones;
    integer i;
```

```
    begin
        ker_flat_s8x9 = 72'd0;
        for (i = 0; i < 9; i = i + 1) begin
            ker_flat_s8x9[i*8 +: 8] = 8'sd1;
        end
    end
endtask

task automatic wait_done_and_check;
    input integer case_id;
    input integer expect_cycles;
    input integer dut_sel;
    integer cycles;
    reg done_now;
    reg [31:0] out_now;
    begin
        begin : wait_loop
            cycles = 1;
            done_now = 1'b0;
            out_now = 32'd0;

            while (1'b1) begin
                case (dut_sel)
                    1: begin
                        done_now = done_base;
                        out_now = out_base_u8x4;
                    end
                    2: begin
                        done_now = done_booth;
                        out_now = out_booth_u8x4;
                    end
                    3: begin
                        done_now = done_simd;
                        out_now = out_simd_u8x4;
                    end
                    4: begin
                        done_now = done_pipe;
                        out_now = out_pipe_u8x4;
                    end
                    default: begin
                        done_now = done_dsp;
                        out_now = out_dsp_u8x4;
                    end
                endcase

                if (done_now === 1'b1) begin
```

```
                    if (cycles !== expect_cycles) begin
                        $display("CASE=%0d FAIL cycles=%0d
                        ↪  exp_cycles=%0d",
                                 case_id, cycles, expect_cycles);
                        $finish;
                    end

                    if (out_now !== EXPECT) begin
                        $display("CASE=%0d FAIL out=%h exp=%h", case_id,
                        ↪  out_now, EXPECT);
                        $finish;
                    end

                    $display("CASE=%0d PASS cycles=%0d out=%h", case_id,
                    ↪  cycles, out_now);
                    @(posedge clk);
                    disable wait_loop;
                end

                @(posedge clk);
                cycles = cycles + 1;
                if (cycles > 200) begin
                    $display("CASE=%0d TIMEOUT", case_id);
                    $finish;
                end
            end
        end
    end
endtask

initial begin
    clk = 1'b0;
    rst_n = 1'b0;
    start_base = 1'b0;
    start_booth = 1'b0;
    start_simd = 1'b0;
    start_pipe = 1'b0;
    start_dsp = 1'b0;
    load_img_1_to_25();
    load_kernel_all_ones();

    repeat(4) @(posedge clk);
    rst_n = 1'b1;
    @(posedge clk);

    start_base = 1'b1;
```

```
        @(posedge clk);
        start_base = 1'b0;
        wait_done_and_check(1, 96, 1);

        start_booth = 1'b1;
        @(posedge clk);
        start_booth = 1'b0;
        wait_done_and_check(2, 96, 2);

        start_simd = 1'b1;
        @(posedge clk);
        start_simd = 1'b0;
        wait_done_and_check(3, 1, 3);

        start_pipe = 1'b1;
        @(posedge clk);
        start_pipe = 1'b0;
        wait_done_and_check(4, 3, 4);

        start_dsp = 1'b1;
        @(posedge clk);
        start_dsp = 1'b0;
        wait_done_and_check(5, 3, 5);

        $display("ALL TESTS PASSED.");
        $finish;
    end
endmodule
```

1.4 基础模块系统波形

图 1 展示了串行顶层的 start/done 时序，可以直观看到 96 周期的延迟行为。

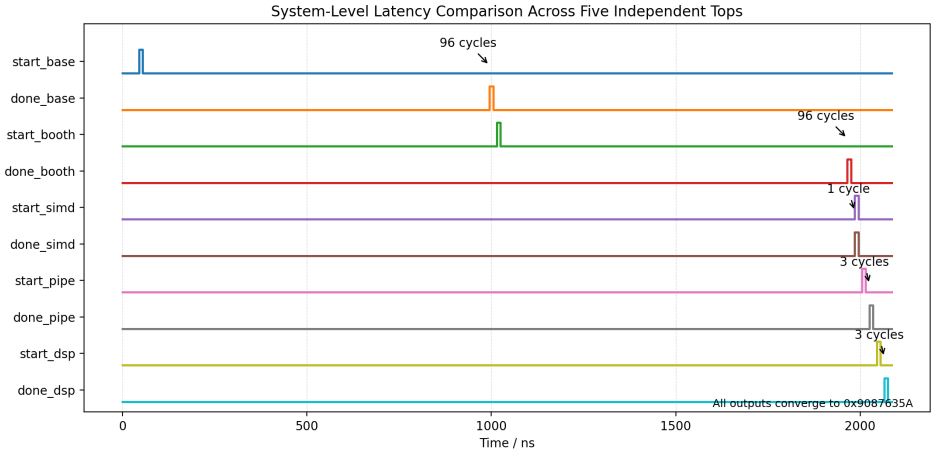

图 1: 系统级波形：串行顶层的 `start`/`done` 延迟